

Voxel Raycasting with 3D DDA

A rewrite guide for project_x

Why rewrite

The original raycaster casts rays through a 3D voxel grid by running three nearly-identical search loops side by side: one for x-aligned grid planes, one for y-aligned, one for z-aligned. Each search marches the ray forward by one voxel-width per step, checks whether that landing spot is inside a solid voxel, and records the distance. The shortest of the three distances wins.

That structure works in principle but is bug-prone in practice. The three search blocks are duplicated code with subtle sign and offset differences, and most of the bugs we hunted lived in those differences: *"this version of the search has a flipped sign relative to the others."* It also relies on a scaffolding of trigonometric "conversion factors" (`xfunc_x`, `zfunc_y`, etc.) that are really just *directional cosine ratios* in disguise: ratios like dx/dy that fall out trivially from a unit direction vector.

The 3D DDA (Digital Differential Analyzer) algorithm replaces all of that with one short loop. It tracks the ray's next crossing of an x-plane, y-plane, and z-plane simultaneously, and at each iteration advances whichever crossing is closest. There is exactly one search, no sign flips, no `round_block_base` off-by-one, no `factor = -1` hack. Step-overshoot at edges (the black gaps between colored faces) is mathematically impossible because the algorithm steps to a grid plane, never past one.

What disappears

- The three duplicated search blocks for x, y, z planes (~250 lines collapse to ~30).
- All trig inside the per-pixel inner loop. Trig appears at most once per ray, in setting up the camera basis, and it is straightforward to eliminate that too once you store the camera as direction vectors.
- The `terminal()` angle-wrap function. Vectors carry direction information; nothing wraps.
- `round_block_base` and the floor-vs-round confusion. DDA computes the next grid line directly from the ray equation.
- The squared-distance vs. true-distance ambiguity in `dist()`. With a unit direction vector, the parameter `t` itself is the true distance.
- The `factor = -1` Z-march hack and the looking up / looking down branches. The step direction in each axis is just the sign of the corresponding direction-vector component.
- The arbitrary `MAX_RAY_DEPTH = 8` iteration cap. DDA reaches any voxel in the minimum possible number of voxel transitions; an upper bound of `mapSize * 3` is generous.

Projection: orthographic vs. perspective

A 3D scene is a set of points in space. A 2D image is a grid of pixels. **Projection** is the rule that says "for each pixel in the image, which point in 3D space does it correspond to?" Equivalently: for each pixel, cast a ray into the scene; whatever the ray hits is what shows up in that pixel. The two projection types differ in *what those rays look like*.

Orthographic

All rays are parallel. From every pixel of the screen, a ray shoots in the same direction. Two identical cubes, one near and one far, render at the same size — there is no perspective shrinkage with distance. Engineering drawings and isometric games use this projection.

The "camera" in orthographic does not really have a position in the usual sense. It has an *orientation* (which way the parallel rays point) and a *viewing rectangle* (where the rays start).

Perspective

All rays start from a single point and fan out. There is one eye-point C. From C, a ray is cast through every pixel of the image plane. Pixels at different positions on the image cast rays in different directions, all originating from C. Far objects render smaller because the same physical-world distance subtends a smaller angle from C. This is how human vision, cameras, and most 3D games work. It is what `GamePanel`'s code is trying to do — `player_x`, `player_y`, `player_z` is the eye-point, and each ray in the inner loop corresponds to one pixel of output, fanning out from that eye-point.

A useful mental picture

Picture the eye-point as a small dot, and the image plane as a flat rectangle floating in front of the dot, like a window. To render the pixel at coordinate (`col`, `row`) on the window, draw a line from the dot, *through* that pixel of the window, and out into the scene beyond. Whatever that line first hits is what appears in that pixel. In orthographic, replace the dot with a *whole flat plane* of dots, all the same distance behind the window — every pixel of the window has its own dot directly behind it, and the ray goes from that pixel-specific dot perpendicularly through the window into the scene. All rays parallel.

Hallway intuition

Imagine standing in a long hallway with parallel walls. With your real eyes, the walls appear to converge toward a point in the distance — they get visually closer together the farther they are. That is perspective. With an orthographic camera, the same hallway would render with the walls *exactly parallel* — as wide at the far end as at the near end. It would look strange, because your eyes are perspective devices. The reason walls converge in perspective is purely geometric: the *angle subtended* by an object shrinks with distance. Two parallel walls ten meters apart, viewed from five meters away, subtend a wide angle from the eye. The same two walls viewed from a hundred meters away subtend a narrow angle. Smaller angle → fewer pixels of screen real estate → walls "look closer."

Which one is DDA?

Neither. DDA is just "march a ray through a voxel grid." It does not care where the ray came from or why. You hand DDA a starting point P and a direction D, and it tells you the first voxel the ray hits. *You* decide whether to feed it parallel rays (orthographic) or fanning rays (perspective). For perspective: every ray starts at the player position; directions vary per pixel. For orthographic: every ray's direction is the same; starting positions vary per pixel. Same DDA loop in both cases, different setup. For this project we want perspective.

Setting up the rays

Before we get to DDA itself, we need two things per ray: a **starting point** and a **direction vector**. For perspective, the starting point is the same for every ray (the player), and each ray's direction is computed from the screen pixel it corresponds to.

Camera basis vectors

Store the camera's orientation as three orthonormal vectors:

- **forward** — the direction the camera is pointing
- **right** — perpendicular to forward, pointing to the camera's right (in the horizontal plane)
- **up** — perpendicular to both, pointing to the camera's up

From the existing `theta` and `epsilon` angles you can compute these once at the start of `drawRay`:

```
forward.x = cos(theta) * cos(epsilon)
```

```

forward.y = sin(theta) * cos(epsilon)
forward.z = sin(epsilon)

right.x   = sin(theta)           // 90° clockwise from forward in the xy-plane
right.y   = -cos(theta)
right.z   = 0

up = cross(right, forward)      // automatically perpendicular to both

```

After this, `theta` and `epsilon` are not used inside the per-pixel loop. The trig is paid once per frame, not 4096 times.

Per-pixel ray direction

For screen pixel at column `col` and row `row` (each running 0 to `VIEW_PIXELS - 1`), compute the ray's direction as a linear combination of the basis vectors:

```

u = (col - VIEW_PIXELS/2) * angle_per_pixel
v = (row - VIEW_PIXELS/2) * angle_per_pixel

```

```

direction = normalize(forward + u * right + v * up)

```

`angle_per_pixel` controls the field of view. With 64 pixels across and 1° per pixel, you get a 64° horizontal FOV. Set it as `FOV_RADIANS / VIEW_PIXELS` for a chosen FOV.

A note on what this is doing geometrically: `forward` points the ray straight ahead, `u * right` nudges it left or right depending on which column we are on, `v * up` nudges it up or down depending on row. Then we `normalize` so $|\text{direction}| = 1$ — this is what makes the DDA parameter `t` equal the true distance to the hit. (If the direction vector had a different length, `t` would be in units of "direction-vector lengths" instead of world units.)

This produces correct rectilinear perspective — the same projection a real camera makes. The "rotate ray angle by DEG per pixel" approach in the original code produces a slightly spherical/fisheye frustum, which is one reason cube edges currently look subtly bowed.

The DDA algorithm

Setup before the loop

Given starting point $P = (px, py, pz)$ and unit direction $D = (dx, dy, dz)$, and a voxel size `VOX` (= 64 in this project), compute:

```

// Which voxel are we in right now?
map_x = floor(px / VOX)
map_y = floor(py / VOX)
map_z = floor(pz / VOX)

// When we cross the next x-plane, does map_x go up or down?
step_x = (dx > 0) ? +1 : -1
step_y = (dy > 0) ? +1 : -1
step_z = (dz > 0) ? +1 : -1

// How much does t increase per voxel-width of motion in each axis?
// (i.e. when the ray's x has moved by VOX, how much t was that?)
t_delta_x = VOX / |dx|
t_delta_y = VOX / |dy|

```

```

t_delta_z = VOX / |dz|

// At what t-value does the ray first cross an x-plane? y-plane? z-plane?
// next_x_line = pixel coordinate of the next x-grid-line in the +step_x direction
if step_x == +1:
    next_x_line = (map_x + 1) * VOX
else:
    next_x_line = map_x * VOX
t_max_x = (next_x_line - px) / dx
// (same for y and z)

```

A subtle but important detail: $t_{\text{max_x}}$ is always positive after this computation. If $dx > 0$, then $\text{next_x_line} > px$ and $dx > 0$, so the ratio is positive. If $dx < 0$, then $\text{next_x_line} < px$ and $dx < 0$, so the ratio is again positive. The signs cancel.

Edge case: $dx == 0$. If the ray has no x-component, it never crosses an x-plane and $t_{\text{max_x}}$ should be infinity. Same for y and z. In code, set $t_{\text{max_x}} = \text{Double.POSITIVE_INFINITY}$ and $t_{\text{delta_x}} = \text{Double.POSITIVE_INFINITY}$ when $dx == 0$. This makes the axis "lose" every comparison, which is correct: that axis never advances.

The loop

```

for (iteration = 0; iteration < MAX_STEPS; iteration++):
    if t_max_x is the smallest of the three:
        map_x += step_x
        t_hit = t_max_x           // distance to the plane we just crossed
        t_max_x += t_delta_x     // next x-plane crossing is one voxel further
        hit_axis = X

    else if t_max_y is the smallest:
        map_y += step_y
        t_hit = t_max_y
        t_max_y += t_delta_y
        hit_axis = Y

    else:
        map_z += step_z
        t_hit = t_max_z
        t_max_z += t_delta_z
        hit_axis = Z

    // Is the new voxel out of map bounds? If so, the ray escaped – no hit.
    if map_x, map_y, or map_z is outside [0, mapSize):
        return NO_HIT

    // Is the new voxel solid? If so, we hit it.
    if map[map_z][map_y][map_x] != 0:
        return (t_hit, hit_axis, map[map_z][map_y][map_x])

// Ran out of iterations without hitting anything
return NO_HIT

```

That is the complete search. About 25 lines including the bookkeeping. The reason it works:

- At any moment, `t_max_x`, `t_max_y`, `t_max_z` are the *t*-values at which the ray will *next* cross an *x*-, *y*-, or *z*-aligned grid plane.
- The smallest of those three is the *t*-value at which the ray *next* transitions to a new voxel. Whichever plane that is, we step the corresponding `map_*` counter and advance that axis's `t_max` to its *next* crossing.
- After the step, we are inside a new voxel. Check whether it's solid; if yes, the ray hit it. If no, loop.

Why the bug classes vanish

- **Step-overshoot at edges** (the black gaps between red/green/blue): impossible. We step to each grid plane, never past it. The hit position is always exactly on a grid plane.
- **Sign confusion** in "looking up vs. down": impossible. `step_x = sign(dx)` handles direction automatically. There are no `if (epsilon < 0)` branches.
- **Off-by-one in starting position**: the initial `t_max_x` calculation handles "where's the next grid line in the direction of motion" with one expression, no round vs. floor ambiguity.
- **Three searches disagreeing about hit position**: there is only one search.
- `dist()` returning squared distance: `t_hit` is the distance to the hit (since the direction is a unit vector). No sqrt, no squaring, no broken comparisons.
- `MAX_RAY_DEPTH = 8` being too few: DDA reaches any in-bounds voxel in at most `mapSize * 3` iterations (since each iteration crosses one grid plane and there are at most that many to cross before leaving the map).

Rendering the hit

Once DDA returns (`t_hit`, `hit_axis`, `voxel_id`) for a ray, draw the corresponding screen pixel:

```
if no hit:
```

```
    color = black (or sky color)
```

```
else:
```

```
    base_color = color_for_voxel(voxel_id)           // texture lookup later
    shade = some_function(t_hit)                     // distance fog: dimmer farther
    if hit_axis == X: shade *= 1.0                    // optional per-axis tinting
    if hit_axis == Y: shade *= 0.85
    if hit_axis == Z: shade *= 0.7
    color = base_color * shade
```

```
drawPixel(col, row, color)
```

The per-axis shading multiplier is a classic Minecraft trick: it gives faces facing different cardinal directions slightly different brightness, which makes the geometry pop without needing real lighting. The values 1.0 / 0.85 / 0.7 are arbitrary — pick what looks good.

Where on screen does the pixel go? At column `col` and row `row` of the `VIEW_PIXELS × VIEW_PIXELS` grid, scaled to the on-screen dimensions of the 3D viewport. This is the same as the existing code:

```
pixel_screen_x = VIEWPORT_X_OFFSET + col * (VIEWPORT_WIDTH / VIEW_PIXELS)
pixel_screen_y = VIEWPORT_HEIGHT - row * (VIEWPORT_HEIGHT / VIEW_PIXELS)
pixel_w = VIEWPORT_WIDTH / VIEW_PIXELS
pixel_h = VIEWPORT_HEIGHT / VIEW_PIXELS
g.fillRect(pixel_screen_x, pixel_screen_y, pixel_w, pixel_h)
```

The `VIEWPORT_HEIGHT - row * ...` inverts the *y*-axis so that `row = 0` is the bottom of the screen and increasing `row` goes up — matching the convention in the existing code.

Full pseudocode

Putting it all together, the structure of drawRayDDA is:

```
function drawRayDDA(g):
    // Compute camera basis once per frame
    fwd_x = cos(theta) * cos(epsilon)
    fwd_y = sin(theta) * cos(epsilon)
    fwd_z = sin(epsilon)
    right_x = sin(theta)
    right_y = -cos(theta)
    right_z = 0
    up_x, up_y, up_z = cross(right, fwd)

    angle_per_pixel = FOV_RADIANS / VIEW_PIXELS

    for row in [0, VIEW_PIXELS):
        for col in [0, VIEW_PIXELS):
            // Compute ray direction
            u = (col - VIEW_PIXELS/2) * angle_per_pixel
            v = (row - VIEW_PIXELS/2) * angle_per_pixel
            dx = fwd_x + u*right_x + v*up_x
            dy = fwd_y + u*right_y + v*up_y
            dz = fwd_z + u*right_z + v*up_z
            len = sqrt(dx*dx + dy*dy + dz*dz)
            dx /= len; dy /= len; dz /= len

            // Run DDA
            (hit, t, axis, voxel) = ddaTrace(player_x, player_y, player_z,
                                             dx, dy, dz)

            // Draw pixel
            if not hit:
                color = BLACK
            else:
                color = shadeFor(voxel, axis, t)
            drawScreenPixel(g, col, row, color)

function ddaTrace(px, py, pz, dx, dy, dz):
    // Initial voxel
    map_x = floor(px / VOX)
    map_y = floor(py / VOX)
    map_z = floor(pz / VOX)

    // Step direction per axis
    step_x = (dx > 0) ? +1 : -1
    step_y = (dy > 0) ? +1 : -1
    step_z = (dz > 0) ? +1 : -1
```

```

// Per-axis t-delta and initial t-max
if dx == 0:
    t_delta_x = INF
    t_max_x = INF
else:
    t_delta_x = VOX / abs(dx)
    next_x_line = (step_x > 0) ? (map_x + 1)*VOX : map_x*VOX
    t_max_x = (next_x_line - px) / dx
// ... same for y and z

for i in [0, MAX_STEPS):
    if t_max_x < t_max_y and t_max_x < t_max_z:
        map_x += step_x
        t_hit = t_max_x
        t_max_x += t_delta_x
        axis = X
    else if t_max_y < t_max_z:
        map_y += step_y
        t_hit = t_max_y
        t_max_y += t_delta_y
        axis = Y
    else:
        map_z += step_z
        t_hit = t_max_z
        t_max_z += t_delta_z
        axis = Z

    if map_x not in [0, mapSize) or
       map_y not in [0, mapSize) or
       map_z not in [0, mapSize):
        return (false, _, _, _)

    if map[map_z][map_y][map_x] != 0:
        return (true, t_hit, axis, map[map_z][map_y][map_x])

return (false, _, _, _)

```

Suggested order of work

Don't try to retrofit DDA into the existing drawRay incrementally. The two algorithms are too structurally different. Instead, add a new method alongside the old one and switch paintComponent once it works. Build it in this order:

1. **Camera basis vectors.** Compute forward, right, up once at the start of the new method. Don't worry about per-pixel rays yet.
2. **Center ray only.** Compute the direction for the single center pixel (col = VIEW_PIXELS/2, row = VIEW_PIXELS/2). Print (dx, dy, dz) to verify it matches the camera's forward direction within rounding.

3. **DDA for one ray.** Implement `ddaTrace` and call it for the center ray only. Print whether it hits, and if so, the `t`, `axis`, and voxel id. Stand near the test cube and verify the values look right (e.g. center ray pointing roughly at the cube returns a hit with `t` around the actual distance to the cube).
4. **Render center ray.** Color the center pixel based on the hit. You'll have a single colored pixel in the middle of the 3D viewport that responds to camera motion.
5. **All rays.** Wrap in the double for-loop over `row` and `col`. The full cube renders.
6. **Polish.** Distance shading, per-axis shading, sky color, etc.

At step 5 you should already see a clean cube with no edge gaps and consistent face colors. If something looks wrong, the most common culprit is a sign or normalization error in step 1 or 2 — verify the basis vectors are orthonormal ($|\text{forward}| = |\text{right}| = |\text{up}| = 1$, dot products zero) before suspecting DDA itself.

Cleanup once it works

Once `drawRayDDA` is rendering correctly, delete the old machinery. The following can all be removed:

- The original `drawRay` method
- `terminal()` (no longer needed; vectors carry direction)
- `round_block_base()`
- `dist()` (DDA returns `t` directly)
- All the `xfunc_*`, `yfunc_*`, `zfunc_*` variables
- The triplet hit-storage variables (`lx`, `ly`, `lz`, `wx`, `wy`, `wz`, `hx`, `hy`, `hz`)
- `MAX_RAY_DEPTH`, `ACCURACY_CONST`, and other accumulated cruft

Net code change: roughly 350 lines deleted, 120 lines added. Cleaner, faster, correct.

Extensions worth considering

Once DDA is working, the following are short, satisfying additions:

- **Higher resolution.** Bump `VIEW_PIXELS` to 128 or 256. Each doubling is 4× the work, but at this map size and DDA's efficiency, 256×256 should still run smoothly.
- **Use** `mountainmap` or `pillarmap` from `Map.java`. These were 9th-grade-you's actual rendering targets and have never been seen rendered.
- **Texture mapping.** When the ray hits a face, compute where on the face it landed (`hit_x`, `hit_y`, `hit_z` modulo `VOX`) and use those to look up a texel in a 2D texture. The existing `Texture.java` already has texture data prepared.
- **Skybox / floor.** When DDA returns no hit, color the pixel based on the ray's direction (e.g. brighter blue if the ray points up, darker if it points down).
- **GPU / compute shader version.** Voxel raycasting is embarrassingly parallel — one thread per pixel. A WebGPU or CUDA port would render at very high resolution in real time. This is a significant project but a natural follow-up.